

Capítulo 7 — GUÍA DE LECTURA

Capítulo 7 — GUÍA DE LECTURA.....	1
Herramientas y componentes: Vite, Web Components y Chart.js.....	2
El empaquetado, el ciclo de vida y el gráfico que no se destruye, explicados en criollo	2
Antes de empezar: cómo usar esta guía.....	2
7.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
La distinción del enunciado, y por qué es precisa	3
7.2 — Por qué hizo falta empaquetar	4
Qué dice	4
En criollo: primero el dolor, después la herramienta	4
El costo apareció con el tamaño, y ahí duele	5
7.3 — Vite	6
7.3.1 — En desarrollo: nadie empaqueta tu código	6
7.3.2 — En producción: acá sí se empaqueta	8
7.3.3 — Vite no verifica los tipos: los borra	8
7.3.4 — Variables de entorno, y la regla del prefijo.....	9
7.4 — Tailwind en el proyecto	9
7.5 — Por qué existen los componentes	10
Qué dice	10
En criollo: los tres puntos donde esa convención se rompía	10
7.6 — Elementos personalizados	10
7.6.1 — Definir y registrar.....	10
Los dos requisitos que la especificación impone.....	11
7.6.2 — El ciclo de vida	11
Dos precisiones que evitan errores concretos	12
7.6.3 — Atributos observados.....	12
7.6.4 — Atributos y propiedades, otra vez	13
7.7 — RN-F01 en su lugar definitivo.....	13
Leé de nuevo la segunda mitad, porque ahí está lo importante.....	13
Por qué el garante hace tres ciclos y no uno.....	14
7.8 — El DOM en la sombra	15
7.8.1 — Qué aísla.....	15
7.8.2 — Ranuras	16
7.8.3 — Lo que complica, y la decisión que te toca tomar	16
7.9 — Plantillas.....	17
Las dos ventajas, y cuál importa de verdad	17

Y una tercera, que para este módulo importa más que las otras dos	18
7.10 — Chart.js y la regla RN-F05.....	18
Por qué hace falta destruir, que no es evidente	18
7.11 — Herramientas de diagnóstico.....	19
7.12 — Seguridad y evolución.....	20
7.13 — Verificación: el checklist honesto.....	20
7.14 — Los doce errores frecuentes	21
7.15 — Las actividades, y qué busca cada una	21
1. Desarrollo contra producción, medido	21
2. La verificación que falta	22
3. Un componente completo	22
4. La clase base	22
5. TST-45 a mano	22
6. Exploración: la frontera de la sombra.....	22
7. Exploración: qué quedó afuera del paquete	22
7.16 — Síntesis: las once frases	22
7.17 — Qué leer, y en qué orden	23
Si lees una sola cosa.....	23
Si lees tres	23
Las fuentes normativas (para consultar, no para leer de corrido)	23
Cierre: las siete cosas que hay que recordar	24

Herramientas y componentes: Vite, Web Components y Chart.js

El empaquetado, el ciclo de vida y el gráfico que no se destruye, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce, y **no se pierde ni un concepto**. Cada sección conceptual tiene tres partes: **Qué dice**, **En criollo** —con la analogía que la hace pegar— y **Para el pizarrón**. En las operativas se va directo.

LA IDEA MADRE DE TODO EL CAPÍTULO

Si te quedás con una sola frase, que sea esta:

Lo que no podés delegar es el ciclo de vida, porque ahí viven todas las reglas.

Este capítulo te enseña a **convertir tu código en algo que un navegador pueda ejecutar** y a **empaquetarlo en piezas que tienen un nacimiento y una muerte**: los dos problemas del día en que tu proyecto deja de ser un archivo y pasa a ser un sistema. Usá todas las bibliotecas que quieras, pero **cuándo se monta y cuándo se desmonta un componente lo manejas vos**.

7.1 — De qué se trata esta clase

Qué dice

El capítulo enseña a convertir el código en algo ejecutable por un navegador y a empaquetarlo en piezas reutilizables con ciclo de vida propio. Conviene abrir deshaciendo un malentendido: el TPI dice **sin framework de interfaz**, y eso suele leerse como «sin dependencias». Es falso: la sección 1.3 declara **doce tecnologías**, varias de ellas bibliotecas de peso.

En criollo

Empecemos por el malentendido, que es el que más proyectos arruina en la primera semana: alguien lee «sin framework de interfaz», entiende «a mano y sin nada», y se pone a escribir su propio cliente HTTP. La lista completa de la sección 1.3 del TPI es esta:

Tecnología	Versión	Rol
TypeScript	5.x	Lenguaje único del cliente, tipado estricto
Vite	5.x	Construcción y servidor de desarrollo, plantilla <code>vanilla-ts</code>
DOM API + Web Components	nativo	Interfaz, componentes y ciclo de vida
History API	nativo	Enrutamiento del lado del cliente
EventSource	nativo	Consumo de los eventos de la sección 11
Tailwind CSS	3.x	Estilos <i>utility-first</i>
<code>@tanstack/query-core</code>	5.x	Estado del servidor: consulta, caché e invalidación
<code>@tanstack/form-core</code>	0.x	Estado y validación de formularios
<code>zustand/vanilla</code>	4.x	Estado del cliente: carrito, sesión e interfaz
Axios	1.x	Cliente HTTP con interceptores
Chart.js	4.x	Gráficos del panel de administración
DOMPurify	3.x	Sanitizador, único mecanismo admitido por RN-F02

Contá: son doce, y ninguna es «vanilla».

La distinción del enunciado, y por qué es precisa

La prohibición no dice «sin bibliotecas»: dice **sin framework de interfaz**. React o Vue **se hacen cargo del ciclo de vida** —cuándo se monta un componente, cuándo se desmonta, cuándo se actualiza— detrás de su maquinaria. Las demás resuelven un problema puntual sin tocar ese ciclo: Axios manda peticiones, Zustand guarda estado, Chart.js dibuja.

Y eso es lo que no podés delegar, porque es donde viven las once reglas. Acá ese ciclo aparece por primera vez de forma explícita: `connectedCallback` y `disconnectedCallback` son los métodos que RN-F01 nombra con esas letras.

Dos reglas se fundan acá —RN-F01 y RN-F05—, y las dos tienen el mismo garante, que no es una recomendación sino un caso de prueba: **TST-45, que monta y desmonta una vista tres veces y verifica que el store quede con cero suscriptores.**

PARA EL PIZARRÓN

Al terminar la clase tenés que poder hacer tres cosas medibles:

1. Crear un proyecto con la plantilla que el TPI declara —`vanilla-ts`—.
2. Escribir un componente que se monte y **se desmonte sin dejar nada atrás**: cero suscripciones vivas, cero manejadores colgados, cero gráficos huérfanos.
3. Explicar por qué tu aplicación puede arrancar perfecta y tener errores de tipo adentro.

7.2 — Por qué hizo falta empaquetar

Qué dice

En 2005, incluir código en una página consistía en escribir etiquetas de script en el orden correcto. Ese «en el orden correcto» era el problema entero: no había módulos en el lenguaje, todo compartía el ámbito global, y el orden era la única forma de expresar una dependencia. Una biblioteca en el lugar equivocado rompía el sitio, sin ningún error.

En criollo: primero el dolor, después la herramienta

Pará antes de leer «Vite»: si empezás por ahí no vas a entender nada.

Imaginate treinta archivos `.js` y que la única forma de decir «este necesita a aquel otro» sea **ponerlos en orden dentro del HTML**. Si movés una línea el sitio se rompe y te dice `undefined is not a function` en una línea que no tiene nada que ver. Y si dos archivos declaran `config`, gana el último. Se probaron varias salidas —entre ellas agrupar todo bajo un único objeto global, que sólo movía el problema de lugar— y ninguna resolvió el fondo hasta la última:

La estrategia	Qué resolvía	Qué seguía roto
Función que se invoca a sí misma	Ámbito privado por archivo	Las dependencias seguían siendo globales: hacía falta el orden correcto
CommonJS · Node.js, 2009	Módulos de verdad: <code>require</code> y <code>module.exports</code>	No servía en el navegador: <code>require</code> es síncrono y pedir por red no lo es
AMD · 2011	Lo asincrónico, para el navegador	Partió la comunidad en dos formatos
Browserify · 2011	Resolver las dependencias antes de que el código llegue al navegador	Un paso de construcción que empezó a tardar
Webpack · 2012	Estilos, imágenes y tipografías en el mismo grafo	Ese paso creció con el proyecto
Rollup · 2015	Código muerto eliminado sobre módulos ES	Una mejora, no otro modelo

Para dibujar el grafo de dependencias

El «árbol de dependencias» es **la receta que te manda a otra receta**: para la salsa necesitás el sofrito; para el sofrito, la cebolla cortada. Nadie te da la lista completa: la vas descubriendo.

Un empaquetador recorre todo ese árbol antes de cocinar, y por eso sabe qué ingrediente no usa ninguna receta y no lo compra: eso es la eliminación de código muerto de Rollup, y funciona porque los `import` son estáticos (Capítulo 3).

El costo apareció con el tamaño, y ahí duele

Se había resuelto el problema de las dependencias. **Y se había creado uno de espera.** A mediados de la década era normal **esperar decenas de segundos** para ver un cambio de una línea. Ese es el dolor, y las dos cosas que lo mataron son de 2020. **esbuild**, escrito en Go por Evan Wallace, demostró que las herramientas eran lentas en buena parte **porque estaban escritas en JavaScript**: hacía lo mismo hasta cien veces más rápido. Y **Vite**, de Evan You, en lugar de acelerar el trabajo observó algo más profundo:

Los navegadores ya entienden `import`. ¿Por qué empaquetar en desarrollo?

De ahí salen las cuatro decisiones de diseño de la herramienta que el TPI declara:

Decisión	Qué gana	Qué cuesta
1. Desarrollo y producción son problemas distintos	En desarrollo importa la recarga; en producción, el tamaño. Optimizar cada uno aparte da lo mejor de los dos	Lo que ves en desarrollo no es lo que se publica
2. En desarrollo no se empaqueta	Módulos ES nativos: el navegador pide lo que necesita cuando lo necesita, y el arranque no depende del tamaño del proyecto	Decenas o cientos de peticiones que asustan la primera vez
3. Las dependencias sí se pre-empaquetan, una sola vez	Una biblioteca tiene cientos de módulos internos; sueltos serían cientos de peticiones. Se convierten a un archivo con esbuild y se cachean	Un arranque más lento la primera vez
4. En producción sí se empaqueta, con Rollup	Servir módulos nativos sin agrupar produce una cascada encadenada, peor que un archivo grande en una conexión real	Un paso de construcción por publicación

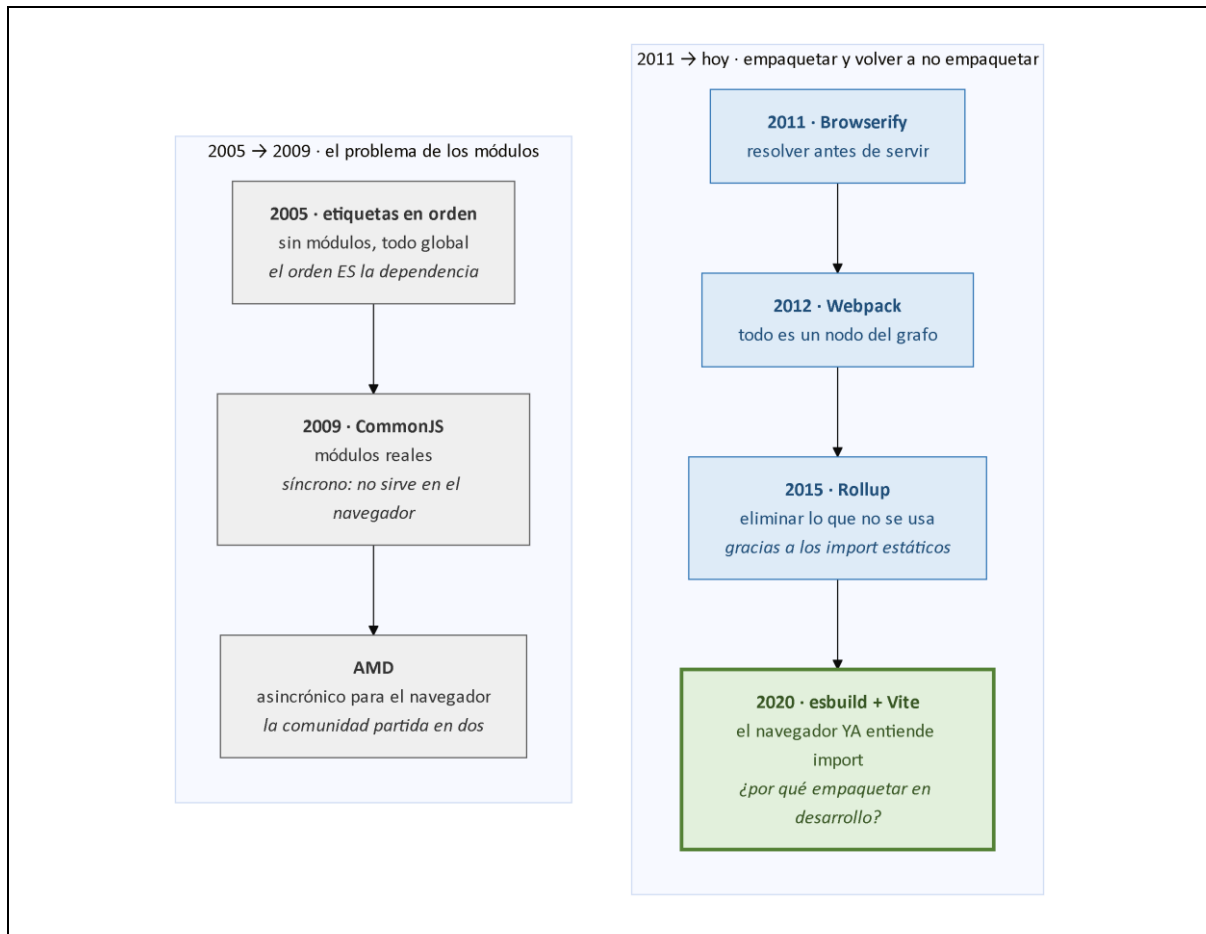


Figura 7.1. Del script suelto al empaquetado

PARA ENTENDER: la pregunta más rentable de la carrera

Es el mismo patrón del Capítulo 4 con jQuery:

Empaquetar en desarrollo era una solución a un problema que dejó de existir.

Cuando Browserify apareció el navegador no entendía módulos. Cuando los módulos ES llegaron a todos los navegadores esa necesidad se evaporó — **pero las herramientas siguieron empaquetando durante años**. Vite no inventó una técnica nueva: **se preguntó por qué se hacía algo, y la respuesta ya no era válida**.

Esa pregunta —*¿qué problema resuelve esto, y sigue existiendo?*— es la que un agente de IA **no** se hace: te propone lo correcto en la mayor cantidad de ejemplos que vio, que suele ser lo correcto hace tres años.

7.3 — Vite

7.3.1 — En desarrollo: nadie empaqueta tu código

Al arrancar, el servidor de desarrollo hace dos cosas, y **ninguna es empaquetar tu código**:

1. **Pre-empaqueta las dependencias** con esbuild y las cachea.
2. **Sirve el código propio como módulos ES**, transformando cada archivo bajo demanda.

El navegador recibe el documento, encuentra el módulo de entrada, y a partir de ahí **pide por sí solo cada import que va encontrando**. Y el servidor transforma **únicamente lo que se pidió**: le quita los tipos de TypeScript, resuelve rutas, responde.

EXPERIMENTO — hacelo hoy, son dos minutos

1. Arrancá el proyecto en desarrollo y abrí el panel de red.
2. Recargá con el panel abierto y **contá las peticiones**.

Vas a ver decenas, quizá cientos: **una por cada archivo .ts de tu proyecto**, más las dependencias pre-procesadas.

1. Hacé clic en una de esas peticiones y mirá la respuesta.

Es tu archivo, con los tipos ya quitados. **El servidor lo transformó recién cuando el navegador lo pidió.**

1. Construí para producción, serví el resultado y contá otra vez: un puñado de archivos, la misma aplicación. Esa diferencia **es** la primera decisión de diseño de la sección 7.2, y las peticiones de desarrollo contra localhost no cuestan casi nada.

La consecuencia es que el tiempo de arranque **no depende del tamaño del proyecto**: uno de diez archivos y uno de mil arrancan casi igual.

Y hay una segunda. Cuando un archivo cambia, la herramienta reemplaza **ese módulo** dentro de la página sin recargar: el carrito lleno sigue lleno. Recargar entero es **cortar la llave general de la casa para cambiar una lámpara**; el reemplazo de módulos en caliente es **cambiar la lámpara con la luz prendida**. Esa es la diferencia entre iterar y sufrir.

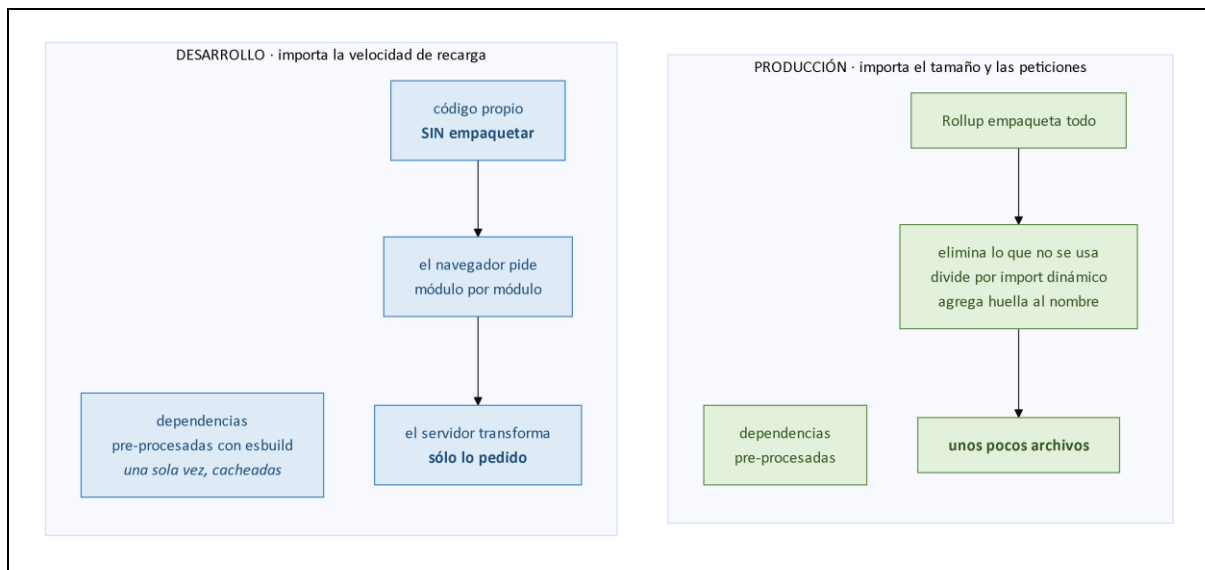


Figura 7.2. Vite: desarrollo frente a producción

FIGURA 7.5

captura

Captura del panel de red en desarrollo, mostrando decenas de peticiones a archivos `.ts`. Idealmente al lado de la misma vista tras construir para producción, para el contraste

Figura 7.5. Módulos ES nativos en desarrollo

7.3.2 — En producción: acá sí se empaqueta

La construcción de producción sí empaqueta, con Rollup, y hace tres cosas más:

- **Elimina lo que no se usa**, porque los `import` son estáticos: se sabe qué se usa sin ejecutar nada. Es la propiedad del Capítulo 3.
- **Divide el resultado en fragmentos**. Cada `import()` dinámico del Capítulo 3 queda en un archivo aparte que se descarga sólo si hace falta: el panel de administración no viaja cuando entra un cliente al catálogo.
- **Agrega una huella al nombre de cada archivo**, que cambia con el contenido y permite cachear agresivamente sin riesgo de servir una versión vieja.

Eso es lo que el TPI declara que sirve el proceso estático de su sección 16.

	En desarrollo	En producción
Tu código	Módulo por módulo, sin empaquetar	Empaquetado con Rollup
Peticiones al cargar	Decenas o cientos	Un puñado
Al guardar	Ese módulo se reemplaza en caliente	Hay que volver a construir
Código no usado	Se sirve igual	Se elimina
Qué se optimiza	La iteración	El tamaño

7.3.3 — Vite no verifica los tipos: los borra

Este punto se anunció en el Capítulo 6 y acá se vuelve operativo. Leelo dos veces:

La herramienta de construcción no verifica los tipos. Los borra.

No es un descuido: **lo hace a propósito**. `esbuild` quita las anotaciones sin analizarlas, y esa es una de las razones por las que es tan rápido; verificar exige construir el grafo de tipos completo, que es otro trabajo. **`esbuild` es el tipógrafo que borra las notas al margen sin leerlas**: nunca prometió corregirte la ortografía.

La consecuencia sorprende a todos la primera vez: **el proyecto arranca perfecto y funciona, con errores de tipo adentro**. Y como arranca igual, es facilísimo convivir con ellos: al mes tenés cuarenta y el editor en rojo dejó de significar algo porque **siempre** está en rojo.

Por eso la verificación es **un paso aparte, explícito, e incluido en el paso de construcción**, para que un error de tipo **impida publicar**. Un proyecto donde la verificación es opcional termina con errores permanentes que nadie mira.

7.3.4 — Variables de entorno, y la regla del prefijo

La configuración que cambia entre desarrollo y producción —la dirección de la API, por ejemplo— se declara en archivos de entorno y se lee mediante `import.meta.env`.

Hasta ahí es comodidad. Lo que sigue no: hay una regla que **no es una convención sino un control de seguridad**. Sólo las variables cuyo nombre empieza con un prefijo declarado se incorporan al código del cliente; las demás quedan afuera.

La razón es directa: **todo lo que llega al cliente es público**, y una variable expuesta queda escrita literalmente en el archivo que el navegador se descarga. El prefijo es **la etiqueta de «apto público» que hay que pegarle a la caja para que suba al camión**: nada sube por accidente.

OJO ACÁ: todo lo que exponés al frontend es público

Todo lo que exponés al frontend es público. Todo. Sin excepción.

No es que «sea difícil de encontrar» ni que «esté ofuscado»: está **escrito en texto plano** dentro de un archivo que cualquiera se baja, y se busca con Ctrl+F.

Una URL de API va bien ahí; una clave pública de un servicio de mapas, también. **Una contraseña de base de datos, una clave de un servicio de pagos o el secreto con el que se firman los tokens, jamás.**

Y pasa siempre igual: alguien copia una variable del `.env` del backend al del frontend «para probar», y queda. Después el repositorio se hace público y la clave queda para siempre en el historial de git. Si te pasa: **no alcanza con borrarla. Hay que rotarla.**

7.4 — Tailwind en el proyecto

El Capítulo 2 explicó qué problema resuelve el enfoque *utility-first* y cómo funciona el escaneo. Acá se agrega lo operativo: la herramienta **se integra al proceso de construcción**, y su configuración declara **qué archivos escanear**.

Ese detalle produce un error difícil de diagnosticar: si un archivo con clases de Tailwind **no está en la lista de rutas escaneadas**, sus clases no se generan. El componente se ve sin estilo, **sin ningún error**.

OJO ACÁ: el noventa por ciento de los «Tailwind no me aplica la clase»

Son dos causas, y ninguna se ve mirando el CSS.

La primera. El archivo no está en las rutas escaneadas: creaste `src/componentes/tarjeta.ts`, la configuración escanea `src/paginas/**`, y esa clase nunca se generó.

La segunda, la de la sección 2.11.2: **el escaneo es textual**. Si armás la clase por concatenación —`"text-" + color`— el escáner ve `"text-"` y una variable, y no genera nada.

Diagnóstico: buscá la clase en el CSS generado. **Si no está ahí, el problema es el escaneo y no el estilo.**

7.5 — Por qué existen los componentes

Qué dice

El Capítulo 4 dejó una carencia planteada sin nombrarla: **el DOM no tiene forma de empaquetar estructura, estilo y comportamiento en una unidad reutilizable**. Un «componente» era una convención: un fragmento de marcado, unas reglas de CSS con prefijo para no colisionar, y una función que registraba manejadores.

En criollo: los tres puntos donde esa convención se rompía

«Convención» quiere decir que funcionaba **mientras todos se acordaran**, y fallaba en tres puntos de los que salen las tres tecnologías de esta sección:

El punto donde fallaba	Qué se rompía	Qué pieza lo resuelve
Los estilos escapaban	Por la cascada del Capítulo 2, escribías <code>.titulo { }</code> para tu tarjeta y se pintaban los títulos de toda la aplicación	El DOM en la sombra
No había ciclo de vida	Nada avisaba cuándo un fragmento entraba o salía del documento, y la baja del Capítulo 4 dependía de que el programador se acordara	Los elementos personalizados
No se podía declarar en el marcado	Había que invocarlo desde código, no escribirlo como un <code><button></code>	Los elementos personalizados

Los frameworks resolvieron eso por su cuenta, cada uno con su modelo. La plataforma respondió con las especificaciones conocidas como **componentes web**, propuestas alrededor de 2011 y estabilizadas en la década siguiente: tres piezas **independientes** —retené esto, que vuelve en la sección 7.8.3—.

Pieza	Resuelve
Elementos personalizados	Declarar un elemento propio, con ciclo de vida
DOM en la sombra	Aislar estructura y estilos
Plantillas	Marcado inerte, listo para clonar

Hubo una cuarta —importar fragmentos de HTML— que **se abandonó**; su reemplazo son los módulos ES del Capítulo 3. Vale mencionarlo porque explica por qué la documentación vieja y **los agentes de IA** siguen nombrando una pieza que ya no existe.

7.6 — Elementos personalizados

7.6.1 — Definir y registrar

```
class TarjetaProducto extends HTMLElement {
  connectedCallback() {
    this.textContent = this.getAttribute("nombre") ?? "";
  }
}

customElements.define("fs-tarjeta-producto", TarjetaProducto);
```

Y a partir de ahí lo escribís en el marcado como cualquier etiqueta nativa:

```
<fs-tarjeta-producto nombre="Milanesa napolitana"></fs-tarjeta-producto>
```

Los dos requisitos que la especificación impone

El nombre debe llevar un guion. No es estética: los elementos nativos **jamás llevan guion**, así que un nombre con guion no puede colisionar con ninguno que el estándar incorpore en el futuro. Y la clase debe extender `HTMLElement`, que habilita los métodos del ciclo de vida. El prefijo `fs-` —por Food Store— no lo pide la norma: evita choques con bibliotecas.

OJO ACÁ: el componente que no falla y no hace nada

Un elemento personalizado que no está registrado no falla. Simplemente no hace nada.

El navegador lo trata como desconocido, lo mete en el árbol como si fuera un `<span>`, y sigue: tu componente aparece vacío, **sin ningún error en la consola**. Es el socio que pasa el molinete porque está en la lista, pero no tiene ficha y nadie le da nada.

Las tres causas, en orden de frecuencia:

1. El módulo que llama a `customElements.define` nunca se importó. Como los `import` son estáticos (Capítulo 3), si nadie lo importa ese código **no existe** en el paquete.
2. El nombre del marcado no coincide con el registrado.
3. El marcado se parseó antes del registro — se resuelve solo cuando el registro llega, pero si nunca llega, nunca se resuelve.

Diagnóstico, desde la consola: `customElements.get("fs-tarjeta-producto")`. Si devuelve `undefined`, **el problema es el registro y no tu componente**.

7.6.2 — El ciclo de vida

Cuatro métodos —cinco con el constructor— que el navegador invoca solo:

Método	Cuándo lo llama el navegador
constructor	Al crear la instancia
connectedCallback	Cada vez que se inserta en el documento
disconnectedCallback	Cada vez que se quita del documento
attributeChangedCallback	Al cambiar un atributo observado
adoptedCallback	Al moverse a otro documento

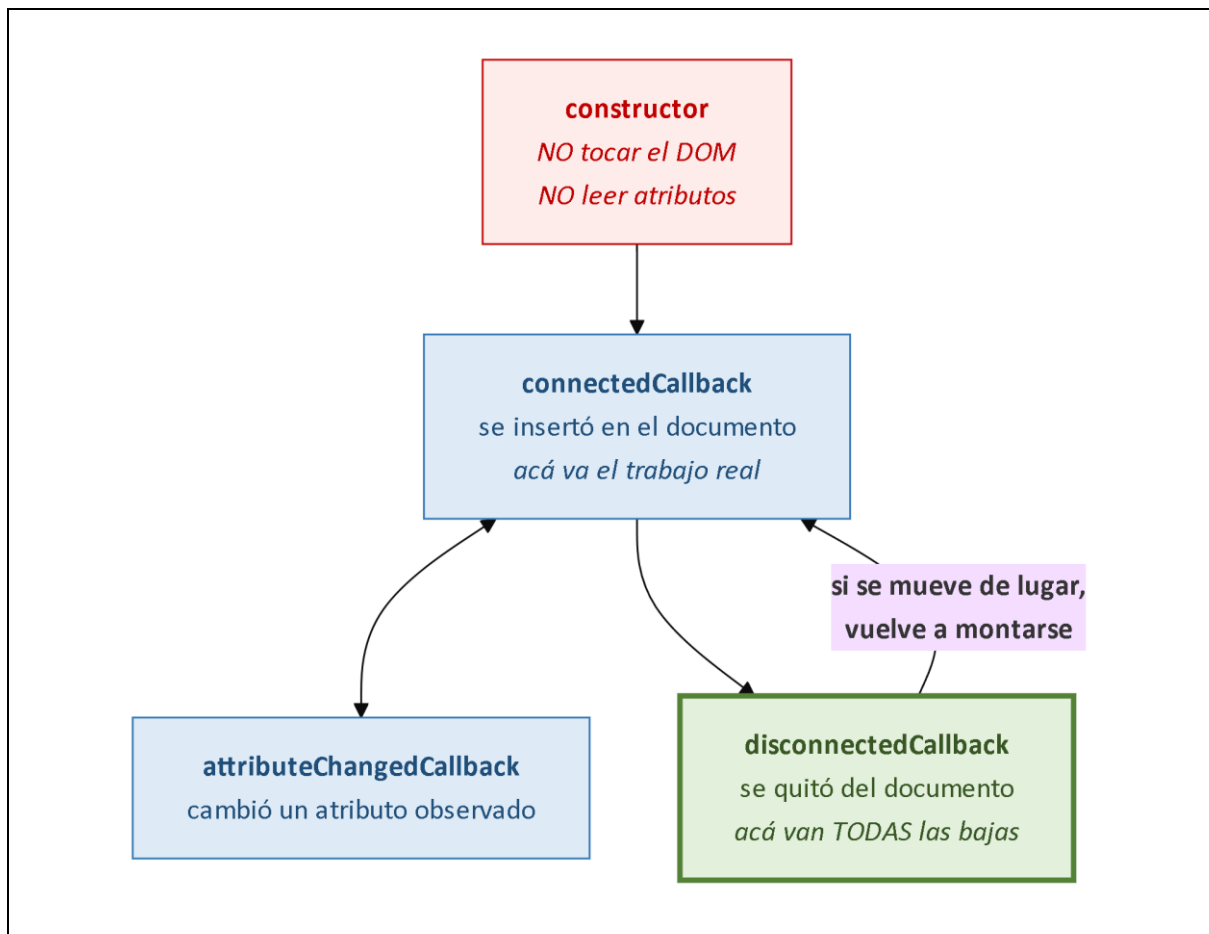


Figura 7.3. El ciclo de vida de un elemento personalizado

Dos precisiones que evitan errores concretos

En el constructor no se toca el DOM ni se leen atributos. El elemento puede existir sin estar completo: sus atributos pueden no haberse aplicado y sus hijos pueden no existir. **La especificación lo prohíbe explícitamente**, y el trabajo real va en `connectedCallback`.

`connectedCallback` puede ejecutarse más de una vez. Si el elemento se mueve de lugar se dispara `disconnectedCallback` y después `connectedCallback` **otra vez**, y el componente que asume un solo montaje **duplica sus suscripciones**: la pantalla se ve igual, el consumo no. Esa flecha de retorno es el punto entero de la Figura 7.3.

7.6.3 — Atributos observados

`attributeChangedCallback` sólo se invoca para los atributos declarados:

```

class EstadoPedido extends HTMLElement {
  static observedAttributes = ["estado"];

  attributeChangedCallback(nombre: string, anterior: string, nuevo: string) {
    if (anterior === nuevo) return;
    this.render();
  }
}
  
```

El `if` de la primera línea no es una optimización menor, y muchos lo borran por «innecesario»: **asignar un atributo con el mismo valor dispara el método igual**. Sin él, un componente que en su `render()` reescribe sus atributos entra en un ciclo.

7.6.4 — Atributos y propiedades, otra vez

La distinción del Capítulo 4 vuelve acá, porque **tu componente tiene que decidir cómo recibe sus datos**.

	Atributos	Propiedades
Qué admiten	Siempre cadenas	Objetos, arreglos, funciones
Para qué sirven	Configuración simple, y declarar el componente en el HTML	Los datos de verdad

Pensalo así: **el atributo es el cartelito colgado en la puerta; la propiedad es lo que hay adentro de la habitación**. El ropero no lo colgás del picaporte.

De ahí sale la consecuencia: **un objeto no se pasa por atributo**, porque serializarlo y volver a parsearlo es frágil y caro. Va por propiedad, desde el código:

```
const tarjeta = document.createElement("fs-tarjeta-producto");
tarjeta.producto = producto; // propiedad: el objeto entero
tarjeta.setAttribute("compacta", ""); // atributo: una bandera
contenedor.appendChild(tarjeta);
```

La tabla de piezas de la sección 2.4 del TPI lo dice al describir la responsabilidad de un componente: **recibe datos**, y expone el ciclo de vida. No los busca por su cuenta.

PARA ENTENDER: «recibe datos» es una regla de arquitectura

El Capítulo 8 va a convertir esa frase en una regla formal: un componente **no llama a la API**. No sabe qué endpoint existe ni si el dato vino de la red o de una prueba. ¿Y por qué conviene? Por tres cosas:

- **Se puede probar** pasándole un objeto, sin levantar ningún servidor.
- **Se puede reusar** en otra pantalla que consiga el dato de otra forma.
- **Cuando algo se ve mal, sabés dónde mirar**: si el dato llegó bien, es el componente; si llegó mal, es la capa de arriba. **Sin esa separación, cada bug se busca en todos lados**.

A un agente: **te va a meter el fetch adentro** si no se lo prohibís, que es exactamente lo que la tabla de piezas del TPI no quiere.

7.7 — RN-F01 en su lugar definitivo

El Capítulo 4 demostró el problema con las manos: una suscripción sin baja produce una fuga que **no se ve mirando la pantalla**. Quedó planteado sin lugar donde poner la solución, porque no teníamos ciclo de vida. Ahora lo tenemos, y el TPI nombra el lugar:

RN-F01. Toda llamada a `subscribe()` devuelve su función de baja; hay que guardarla y ejecutarla en `destroy()` o `disconnectedCallback()`, que es obligatorio. **Ninguna suscripción se hace fuera de la clase base Disposable**, que acumula las bajas y las ejecuta en bloque. Garante: **TST-45**, que monta y desmonta una vista tres veces y verifica que el store quede con cero suscriptores.

Leé de nuevo la segunda mitad, porque ahí está lo importante

La regla no dice sólo «acordate de dar de baja»: dice **que exista una clase base que lo haga por vos**, y que **ninguna suscripción ocurra fuera de ella**. «Acordate» falla el día que alguien está apurado, y en un TPI alguien siempre está apurado. Una clase base que acumula las bajas convierte el olvido en algo **estructuralmente difícil**:

```
abstract class Disposable extends HTMLElement {
```

```
private bajas: Array<() => void> = [];

protected registrar(baja: () => void): void {
  this.bajas.push(baja);
}

disconnectedCallback(): void {
  for (const baja of this.bajas) baja();
  this.bajas = []; // importante: puede volver a montarse (7.6.2)
}
}
```

Esa línea está ahí por la sección 7.6.2: si no vacías la lista, el próximo montaje agrega bajas sobre las viejas y las ejecuta dos veces.

```
class PanelDePedidos extends Disposable {
  connectedCallback() {
    this.registrar(pedidosStore.subscribe(this.alCambiar));
    this.registrar(observador.subscribe(this.alLlegarDatos));

    const c = new AbortController();
    window.addEventListener("resize", this.alRedimensionar, { signal: c.signal });
  };
  this.registrar(() => c.abort()); // el AbortController del Capítulo 4
}
}
```

Fijate en lo que **no** está escrito: `PanelDePedidos` **no define** `disconnectedCallback`. Lo hereda, y no hay nada que olvidar porque **no hay nada que escribir**. Es la **bandeja de las llaves en la entrada**: cada vez que abrís algo dejás ahí la llave de cerrarlo, y al salir cerrás todo junto.

Por qué el garante hace tres ciclos y no uno

TST-45 monta y desmonta una vista **tres veces** y verifica que el store quede con **cero** suscriptores.

Con un solo ciclo, una baja faltante puede pasar desapercibida: un contador en 1 se confunde con cualquier cosa. Con tres, el test falla **con un número que dice qué pasó**. Es una prueba diseñada para detectar la acumulación, no la ausencia.

PARA ENTENDER: la diferencia entre una recomendación y un diseño

«Acordate de dar de baja» es una recomendación; «no podés suscribirte fuera de `Disposable`» es un diseño.

Una recomendación depende de que todos, a las dos de la mañana antes de entregar, se acuerden. Va a fallar — no por descuidados: **porque las recomendaciones siempre fallan a escala**. Un diseño hace que la forma fácil sea la correcta: si suscribirse pasa por `registrar()`, el olvido deja de ser posible sin salirse del camino.

Preguntate en cada regla: *¿estoy confiando en que alguien se acuerde, o estoy haciendo que sea difícil equivocarse?*

Y ojo con los agentes: **te van a escribir el `subscribe` y no la `baja`**. El `subscribe` hace falta para que funcione; la `baja`, para que *siga* funcionando, y eso no aparece en la demo.

7.8 — El DOM en la sombra

7.8.1 — Qué aísla

Un elemento puede tener un subárbol propio, separado del documento principal:

```
class TarjetaProducto extends HTMLElement {
  connectedCallback() {
    const sombra = this.attachShadow({ mode: "open" });
    sombra.innerHTML = `
      <style>
        .titulo { font-weight: 600; } /* no escapa */
      </style>
      <h3 class="titulo"><slot name="nombre"></slot></h3>
    `;
  }
}
```

El aislamiento funciona **en las dos direcciones**: los estilos declarados adentro no afectan al resto del documento, y los del documento no entran. Los selectores tampoco cruzan: un `querySelectorAll(".titulo")` sobre el documento **no encuentra** los elementos que están dentro de una sombra.

Para dibujarlo: **el DOM en la sombra es un local dentro de un shopping**. Le llega el aire acondicionado y la luz general —las propiedades heredables del Capítulo 2— y podés dejarle tomas preparadas para que enchufe lo que quiera —las propiedades personalizadas, previstas para la personalización controlada desde afuera—. Pero **su vidriera es suya**, y **el plano del shopping no lista sus estantes**: no cruzan las reglas de estilo del documento —**incluidas las clases de Tailwind**—, ni los selectores, ni las relaciones de accesibilidad expresadas por identificador.

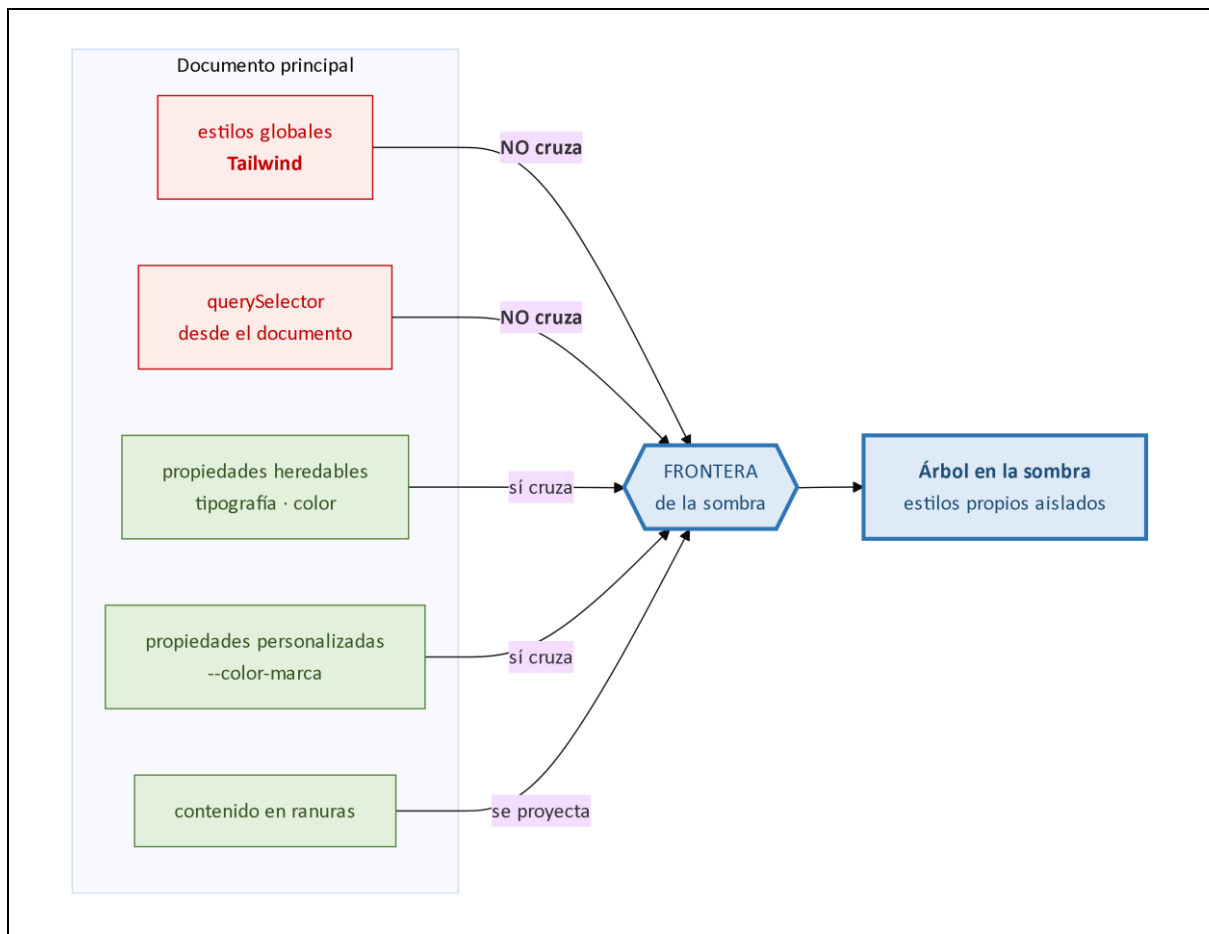


Figura 7.4. Qué aísla el DOM en la sombra

7.8.2 — Ranuras

Una **ranura** es un hueco donde se proyecta el contenido que el usuario del componente escribió entre las etiquetas. Lo que se malinterpreta es de quién sigue siendo: **vive en el documento principal** y por lo tanto **conserva sus estilos**; sólo se muestra donde la ranura indica.

Es el portarretrato: **el marco es del componente, la foto la ponés vos y sigue siendo tuya.**

7.8.3 — Lo que complica, y la decisión que te toca tomar

Las clases de Tailwind no cruzan la frontera. Tailwind genera una hoja de estilos global, y una hoja global no entra en un DOM en la sombra: un componente con sombra que use `class="flex items-center p-4"` **no recibe ninguno de esos estilos.**

Hay formas de sortearlo, todas incómodas: inyectar la hoja en cada sombra, usar hojas adoptables, o duplicar estilos. Y hay una salida simple: **usar elementos personalizados sin DOM en la sombra**, lo que se conoce como quedarse en el DOM claro. Acá vuelve lo de la sección 7.5: **las tres tecnologías son independientes.**

La opción	Qué ganás	Qué perdés
Sin sombra (DOM claro)	Tailwind funciona igual que en cualquier parte, y conservás <code>connectedCallback</code> , <code>disconnectedCallback</code> y el elemento declarado en el marcado: todo lo que las reglas necesitan	El encapsulamiento de estilos, que con Tailwind importa menos porque las utilidades ya acotan el alcance
Con sombra + hoja adoptable	Aislamiento real de estilos y selectores	Complejidad por componente, más los dos costos de accesibilidad
Con sombra y nada más	Nada, acá	Todo Tailwind

Los **dos costos de accesibilidad** conviene conocerlos antes de elegir: **una etiqueta de formulario no puede referenciar por identificador a un campo que está dentro de una sombra**, y ninguna relación de accesibilidad expresada por identificador cruza la frontera. Eso rompe cosas que vos no ves y un lector de pantalla sí.

NOTA: no es una contradicción del enunciado, es una decisión tuya

El TPI declara **Web Components** y declara **Tailwind**, y las dos tienen una fricción real. No es una contradicción: **es una decisión de diseño que te toca tomar a vos**, a conciencia y **una sola vez para todo el proyecto**.

La opción razonable, salvo motivo concreto para lo contrario, es **elementos personalizados sin sombra**: ganás el ciclo de vida y Tailwind sigue funcionando.

Lo que **no** podés hacer es mezclar sin criterio. Vas a terminar con estilos que aplican en la mitad de la pantalla y con horas perdidas buscando un problema de CSS que **es un problema de arquitectura**.

7.9 — Plantillas

El elemento de plantilla contiene marcado que **no se renderiza ni se ejecuta**: sus imágenes no se descargan y sus scripts no corren. Existe para ser clonado. Es el molde, y el molde no se come.

```
<template id="tpl-tarjeta">
  <article class="rounded-lg border p-4">
    <h3 data-campo="nombre"></h3>
    <p data-campo="precio"></p>
  </article>
</template>
const tpl = document.querySelector<HTMLTemplateElement>("#tpl-tarjeta");
const nodo = tpl.content.cloneNode(true) as DocumentFragment;
nodo.querySelector('[data-campo="nombre"]')!.textContent = producto.nombre;
```

Las dos ventajas, y cuál importa de verdad

La primera es que **la estructura queda legible como marcado**, en vez de repartida en quince llamadas a `createElement`.

La segunda es que **clonar es más rápido que crear elemento por elemento**, y hay que ser preciso porque importa recién a cierta escala: con listas de decenas o cientos —un catálogo, un historial de pedidos— se nota; con tres elementos, no. **La razón principal para usar plantillas no es el rendimiento sino la legibilidad**.

Y una tercera, que para este módulo importa más que las otras dos

El contenido de la plantilla es marcado estático que escribiste vos, y los datos se insertan con `textContent`. Nunca hay una cadena de datos que se parsee como marcado. Eso cumple **RN-F02 por construcción**, sin que nadie tenga que recordarlo — el mismo principio que la clase `Disponible` de la sección 7.7.

PARA ENTENDER: el patrón que se repite tres veces en este capítulo

Las tres se podrían haber escrito como advertencias en un README. **Ninguna habría sobrevivido a un cuatrimestre.** Escritas como estructura, no hay nada que recordar: **el camino fácil es el correcto.** Cuando diseñes algo, esa es la pregunta que vale: *¿estoy escribiendo una advertencia o estoy cambiando la forma del camino?*

7.10 — Chart.js y la regla RN-F05

El panel de administración del TPI muestra gráficos con Chart.js, y hay una regla específica para eso:

RN-F05. Toda instancia de Chart.js se crea **una sola vez por montaje** y se destruye con `chart.destroy()` al desmontar; la actualización se hace **mutando `chart.data` y llamando `chart.update()`**. Garante: TST-45, por la misma vía que RN-F01 —la destrucción del gráfico se registra como una baja más.

Esa última frase es la clave del diseño: **el gráfico no es un caso especial.** Su destrucción se registra en el mismo `Disponible` de la sección 7.7, al lado de las suscripciones. **Una sola disciplina cubre las dos reglas.**

```
class GraficoVentas extends Disponible {
  private chart?: Chart;

  connectedCallback() {
    const canvas = this.querySelector("canvas")!;
    this.chart = new Chart(canvas, configuracion);
    this.registrar(() => this.chart?.destroy()); // una baja más

    this.registrar(ventasStore.subscribe(datos => {
      this.chart!.data.datasets[0].data = datos; // mutar
      this.chart!.update(); // y actualizar
    }));
  }
}
```

Por qué hace falta destruir, que no es evidente

Una instancia de Chart.js hace tres cosas por su cuenta: **registra manejadores en la ventana** para redimensionar, **se adueña del elemento de dibujo** y **mantiene sus propias estructuras internas**. Nada de eso se va solo cuando perdés la referencia.

Si creás una instancia nueva sobre el mismo elemento sin destruir la anterior, **las dos quedan vivas**: las dos responden al redimensionado, las dos dibujan sobre el mismo lienzo, y aparecen globos duplicados y animaciones que se pisan, además de la fuga.

Y la segunda mitad de la regla —mutar y actualizar— tiene la misma raíz: **crear una instancia nueva en cada actualización es el error que la primera mitad prohíbe**, repetido muchas veces por segundo.

OJO ACÁ: el error que te va a escribir un agente con total naturalidad

En el panel del TPI —que recibe eventos por SSE, los de la sección 11— se pone feo rápido. Le pedís «que el gráfico se actualice cuando lleguen datos» y te sale esto:

```
ventasStore.subscribe(datos => {
  new Chart(canvas, { ...config, data: datos }); // instancia nueva cada vez
});
```

Anda. Se ve bien. **Y cada evento deja una instancia viva más.** A los cincuenta eventos tenés cincuenta gráficos sobre el mismo lienzo y globos duplicados: cincuenta pintores frente al mismo cuadro. Lo correcto son dos líneas:

```
this.chart.data.datasets[0].data = datos; // mutar
this.chart.update();                       // y actualizar
```

Es **el mismo error del Capítulo 4** con otra cara: algo que se crea y no se destruye, acumulándose sin que la pantalla lo muestre.

7.11 — Herramientas de diagnóstico

En desarrollo, el panel de red muestra **decenas o cientos de peticiones**, una por cada módulo. Es correcto, y no es un problema de rendimiento: **es la decisión de diseño de la sección 7.2 a la vista**. En producción la misma página descarga unos pocos archivos, y comparar las dos vistas es el mejor ejercicio para entender la herramienta.

Para los componentes, el panel de elementos los muestra con su nombre propio —vas a ver `<fs-tarjeta-producto>` tal cual, no un `<div>`—, y cuando tienen sombra aparece un nodo especial desplegable. La consola permite además inspeccionar el registro: `customElements.get("fs-tarjeta-producto")` devuelve la clase si está registrada y `undefined` si no, que es el síntoma de la sección 7.6.1.

FIGURA 7.6

captura

Captura del panel de elementos con un `fs-*` desplegado. Si el proyecto usa sombra, con el nodo de sombra visible; si no, mostrando que el contenido vive en el DOM claro

Figura 7.6. Un elemento personalizado en el inspector

Y para verificar RN-F01, el procedimiento del Capítulo 4 sigue siendo el mismo, con un agregado más directo: **contar los suscriptores del store**, que es lo que hace TST-45.

EXPERIMENTO — reproducí TST-45 a mano antes de escribirlo

1. Escribí un componente que se suscriba a un store en `connectedCallback` **sin** dar de baja.
2. Desde la consola, montalo y desmontalo **tres veces**.
3. Contá los suscriptores del store.

Te van a quedar **tres**: la pantalla está vacía y hay tres componentes fantasma escuchando cambios y actualizando nodos que ya nadie ve.

1. Hacé que el componente herede de `Disposable` y repetí los tres ciclos.
2. Contá otra vez: **cero**.

Ese conteo es literalmente el garante que el TPI declara. **Con tres ciclos, el número te dice exactamente cuántas bajas te olvidaste.**

7.12 — Seguridad y evolución

La primera: todo lo que entra al paquete es público. Vale para las variables de entorno de la sección 7.3.4 y para cualquier constante del código: el paquete se lee, y aunque los nombres estén acortados, **las cadenas de texto quedan intactas.**

La segunda: cada dependencia es código de terceros que corre con los mismos privilegios que el propio. Un paquete comprometido puede leer el token, modificar el DOM o emitir peticiones: por eso la lista de doce tecnologías de la sección 7.1 es corta **a propósito.**

La tercera: un elemento personalizado que arma su contenido con `innerHTML` a partir de datos recibidos reintroduce el problema del Capítulo 4. Y ojo con esta creencia: **el aislamiento del DOM en la sombra no protege de la inyección.** La sombra aísla estilos y selectores, no ejecución. **RN-F02 vale adentro y afuera.**

Sobre la evolución, tres incorporaciones recientes atacan justamente las fricciones de la sección 7.8.3: las **hojas de estilo adoptables** permiten compartir una hoja entre muchas sombras sin duplicarla —la respuesta directa al problema de Tailwind—; los **elementos personalizados asociados a formularios** permiten que un componente participe de un formulario nativo, que era una de las carencias más molestas; y las **consultas de contenedor** del Capítulo 2 hacen que un componente responda a **su contenedor** y no a la ventana, que es exactamente lo que un componente reutilizable necesita.

7.13 — Verificación: el checklist honesto

No son ejercicios: son el criterio para saber si el capítulo se entendió.

- Crear un proyecto con la plantilla que el TPI declara y **contar las peticiones** del panel de red en desarrollo. (7.3.1)
- Construirlo para producción y **volver a contarlas**, explicando la diferencia. (7.3.2)
- Introducir un error de tipo deliberado y **verificar que arranca igual**; ejecutar después la verificación por separado y ver el error. (7.3.3)
- Declarar una variable de entorno con el prefijo requerido y otra sin él, y **buscar las dos en el paquete**. (7.3.4)
- Escribir un elemento personalizado que informe en `connectedCallback` y en `disconnectedCallback`, y **moverlo de lugar** observando la consola. (7.6.2)
- Implementar `Disposable` y un componente que la use, y demostrar que **no define `disconnectedCallback` propio**. (7.7)
- Reproducir TST-45 a mano: montar y desmontar tres veces y **contar suscriptores**. (7.7)
- Crear un componente con sombra que use clases de Tailwind y **verificar que no se aplican**; resolverlo quitando la sombra. (7.8.3)
- Instanciar Chart.js dos veces sobre el mismo lienzo sin destruir, y **documentar los síntomas**. (7.10)

7.14 — Los doce errores frecuentes

Todos tienen algo en común: **en el momento, no parecen errores**, y varios ni siquiera producen un mensaje. Por eso son frecuentes.

El error	Por qué duele	Sección
Creer que «sin framework» significa «sin dependencias»	Son doce tecnologías: lo prohibido es el framework de interfaz	7.1
Suponer que el proyecto arranca porque los tipos están bien	La construcción borra los tipos sin verificarlos	7.3.3
Exponer un secreto en una variable de entorno del cliente	Queda en texto plano en el paquete público, y borrarlo no alcanza: se rota	7.3.4
Olvidar un archivo en las rutas escaneadas de Tailwind	Sus clases no se generan y el componente queda sin estilo, sin error	7.4
Definir un elemento personalizado sin guion en el nombre	El registro falla	7.6.1
Tocar el DOM o leer atributos en el constructor	El elemento puede no estar completo; el trabajo va en <code>connectedCallback</code>	7.6.2
Asumir que <code>connectedCallback</code> se ejecuta una sola vez	Mover el elemento lo redispara y las suscripciones se duplican	7.6.2
Pasar un objeto por atributo	Los atributos son siempre cadenas ; serializar y reparsear es frágil	7.6.4
Suscribirse fuera de la clase base	Viola RN-F01: ninguna suscripción fuera de Disposable	7.7
No vaciar la lista de bajas tras ejecutarlas	Si se vuelve a montar, acumula	7.7
Usar DOM en la sombra con Tailwind sin resolver el aislamiento	Ninguna clase aplica. Parece CSS y es arquitectura	7.8.3
Crear una instancia de Chart.js nueva en cada actualización	Deja instancias superpuestas. Hay que mutar y actualizar	7.10

7.15 — Las actividades, y qué busca cada una

1. Desarrollo contra producción, medido

Crear el proyecto, documentar peticiones y tiempo de arranque en desarrollo, construir para producción y repetir la medición. Explicar cada diferencia a partir de **las cuatro decisiones de diseño de la sección 7.2**.

Qué busca: *que el número reemplace a la intuición.*

2. La verificación que falta

Introducir tres errores de tipo, comprobar que **arranca igual**, y configurar el paso de construcción para que un error de tipo **impida publicar**. Documentar qué comando lo hace.

Qué busca: *ver la aplicación andando con los tipos rotos, y cerrar esa puerta vos.*

3. Un componente completo

Implementar `fs-tarjeta-producto` que reciba el producto **por propiedad**, muestre nombre e importe respetando **RN-F08**, se suscriba al store del carrito y **se dé de baja al desmontarse**. Sin DOM en la sombra.

Qué busca: *juntar en un archivo el capítulo entero: registro, ciclo de vida, propiedad y baja.*

4. La clase base

Implementar `Disponible` con `registrar()` y `disconnectedCallback()`, y refactorizar dos componentes para que la usen. **Verificar que ninguno define el suyo.**

Qué busca: *que no haya nada escrito es la prueba de que el diseño funcionó.*

5. TST-45 a mano

Escribir el procedimiento de conteo de suscriptores, ejecutarlo sobre un componente con fuga y sobre uno correcto, y documentar los dos resultados. **Explicar por qué son tres ciclos y no uno.**

Qué busca: *entender por qué tres es entender que el test mide acumulación.*

6. Exploración: la frontera de la sombra

Construir el mismo componente con y sin DOM en la sombra, usando clases de Tailwind y una regla de estilo global. Documentar qué se aplica en cada caso, qué encuentra un `querySelector` desde el documento y qué pasa con una etiqueta de formulario que referencia un campo interno.

Relacionarlo con la sección 7.8.3 y justificar la elección para el TPI.

Qué busca: *que la decisión de la sección 7.8.3 la tomes vos con datos propios.*

7. Exploración: qué quedó afuera del paquete

Construir para producción y abrir el resultado. Buscar tres cadenas del código propio, una variable de entorno expuesta y una no expuesta, y una función que no se usa. Documentar qué se encontró y qué no, y relacionarlo con **la eliminación de código muerto de la sección 7.3.2** y con **los `import` estáticos del Capítulo 3**.

Qué busca: *comprobar que el paquete es legible, y por lo tanto público.*

7.16 — Síntesis: las once frases

1. **«Sin framework» no es «sin dependencias».** Lo prohibido es el framework de interfaz, porque se haría cargo del ciclo de vida, y **el ciclo de vida es lo que no podés delegar**: ahí viven las reglas.
2. Empaquetar nació de una carencia del lenguaje —no había módulos— y de una limitación del transporte. Cuando los módulos ES llegaron al navegador, **empaquetar en desarrollo pasó a resolver un problema que ya no existía**.
3. Vite separa **desarrollo y producción como problemas distintos**: sin empaquetar en el primero, empaquetado en el segundo, con las dependencias pre-procesadas una sola vez.
4. **La herramienta de construcción no verifica los tipos: los borra.** El proyecto arranca perfecto con errores adentro, y por eso la verificación debe ser un paso explícito de la publicación.

5. **Todo lo que entra al paquete es público**, empezando por las variables de entorno. Un secreto expuesto no se arregla borrándolo: **se rota**.
6. Los componentes web son **tres tecnologías independientes** y se pueden usar por separado. **El ciclo de vida sirve sin el aislamiento**.
7. **connectedCallback puede ejecutarse más de una vez**, y en el constructor no se toca el DOM. Los objetos van por propiedad, porque **los atributos son siempre cadenas**.
8. RN-F01 no dice «acordate de dar de baja»: dice que **ninguna suscripción ocurra fuera de una clase base que lo haga por vos**. Es la diferencia entre una recomendación y un diseño.
9. **TST-45 monta y desmonta tres veces** y cuenta suscriptores. Tres, y no uno, porque la prueba está diseñada para detectar **acumulación**.
10. **Tailwind y el DOM en la sombra tienen una fricción real**: los estilos globales no cruzan la frontera. La decisión razonable —elementos personalizados sin sombra— se toma **una vez y para todo el proyecto**.
11. RN-F05 no trata al gráfico como un caso especial: **su destrucción se registra como una baja más** en el mismo mecanismo de RN-F01. Una sola disciplina cubre las dos reglas.

7.17 — Qué leer, y en qué orden

Si leés una sola cosa

La guía de **web.dev sobre componentes web**. Explica las tres piezas por separado, con ejemplos que corren, y **no te empuja a usar la sombra**, que es justamente la decisión que la sección 7.8.3 te pide tomar a conciencia.

Si leés tres

- La **documentación oficial de Vite**, en `vite.dev`: el pre-procesamiento de dependencias y el reemplazo de módulos en caliente explicados por quien los diseñó, y su **guía de variables de entorno** con la regla del prefijo de la sección 7.3.4.
- La **documentación de esbuild**, sobre todo su sección **por qué es rápido**: explica decisiones de diseño trasladables a otros contextos.
- **Pickering, Inclusive Components** (Smashing Magazine, 2016) —ya citado en el Capítulo 2—: la mejor guía práctica para componentes que funcionan con teclado y lector de pantalla, y **su tratamiento de los costos de accesibilidad del DOM en la sombra complementa la sección 7.8.3**.

Las fuentes normativas (para consultar, no para leer de corrido)

- **HTML Living Standard** (WHATWG): la sección de **elementos personalizados** define el registro, los requisitos del nombre y los cuatro métodos del ciclo de vida de la sección 7.6.2, con la **prohibición explícita de tocar el DOM en el constructor**; la de **plantillas**, el comportamiento inerte de la sección 7.9.
- **DOM Living Standard** (WHATWG): el árbol en la sombra, las ranuras y el modo de aislamiento de la sección 7.8.
- **CSS Scoping Level 1** (W3C): estilos y sombra, con `:host` y `::part()`. **CSSOM Level 1** define las **hojas adoptables** de la sección 7.12.
- **ECMA-262**: los módulos ES que hacen posible el modelo de desarrollo de la sección 7.3.1, con su carga definida en el estándar HTML.
- **Documentación de Chart.js**: sus páginas de **ciclo de vida y actualización de datos** cubren lo que RN-F05 exige. **MDN**, sobre elementos personalizados, es la referencia de consulta cotidiana.
- **Del TPI, la tabla de piezas de la sección 2.4**: declara la responsabilidad de cada archivo del frontend, de ahí sale el «recibe datos» de la sección 7.6.4, y **es el mapa que el Capítulo 8 va a recorrer entero**.

Cierre: las siete cosas que hay que recordar

Si dentro de un mes te acordás de siete frases de todo esto, que sean estas.

LAS SIETE

1. Lo prohibido no son las bibliotecas: **es delegar el ciclo de vida**.
2. Vite no inventó nada: **se preguntó por qué se empaquetaba en desarrollo**, y la respuesta ya no era válida.
3. La construcción **borra los tipos, no los verifica**. Si la verificación no está en la publicación, no existe.
4. Todo lo que entra al paquete **es público y se busca con Ctrl+F**. Un secreto filtrado no se borra: se rota.
5. **connectedCallback se puede ejecutar más de una vez**. Quien asuma lo contrario acumula suscripciones sin que la pantalla lo muestre.
6. RN-F01 no pide memoria, **pide una clase base**: la bandeja de las llaves en la entrada.
7. El gráfico **no es un caso especial**: su `destroy()` es una baja más.

Y una octava, que no está escrita en el capítulo pero está en todas sus páginas: **cuando tengas que hacer cumplir una regla, no escribas una advertencia — cambiá la forma del camino**. Disposable, la plantilla estática y el prefijo de las variables de entorno hacen lo mismo: que olvidarse dé más trabajo que hacerlo bien.

Continúa en: Capítulo 8 — Arquitectura: Feature-Sliced Design, estado y el TPI, donde las piezas de este capítulo se ordenan en capas con una regla de dependencias, y donde el módulo cierra con lo que lo motivó: cómo dirigir a un agente de IA sobre una base que uno entiende.